Distorções, Deformações e Preenchimento de Regiões

Da geometria e das matrizes à aplicação em Python + OpenGL

Computação Gráfica – Aula 17

Material didático complementar

Roteiro: o que vamos construir?

1. **Distorções/deformações:** escala em X e mapeamento por triângulos.
2. **Preenchimento:** 4/8-conectividade, flood fill e borda.

Método da aula

Conceito → previsão → trecho Python → executar OpenGL → interpretar resultados.

Todas as imagens da demonstração são matrizes NumPy; nenhuma depende de arquivos externos.

Revisão opcional: de convolução a bordas

- A convolução combina a vizinhança de cada pixel com **pesos de um kernel**.
- Sobel X e Y estimam mudanças horizontais e verticais de intensidade.
- A magnitude aproximada do gradiente é $\sqrt{G_x^2 + G_y^2}$.
- No programa, o modo **0** compara uma imagem RGB e suas bordas.

Prática de retomada

Se já foi estudada a convolução, demonstre diretamente. Caso contrário, deixe o **TODO 01** como desafio opcional.

O que é distorção (ou deformação) geométrica?

Definição

É a modificação das **posições espaciais** de pontos de um objeto ou imagem por meio de uma transformação T: $(x', y') = T(x, y)$. Dependendo de T, mudam a **forma**, as **proporções** ou a disposição local da imagem.

- **Translação/rotação rígida:** deslocam ou giram, mas não distorcem a forma.
- **Escala não uniforme, cisalhamento, perspectiva:** podem produzir distorção visual.
- **Deformação local:** diferentes regiões podem mudar de formas diferentes.

Pergunta: aumentar apenas a largura mantém as proporções?

Tipos de distorção: duas classificações úteis

Pela abrangência

- **Global:** uma regra para toda a imagem.
- **Local:** a regra muda conforme a região.

Pela dimensionalidade

- **Em um eixo:** modifica x ou y.
- **Em dois eixos:** modifica x e y.

Pela transformação

- **Afim:** escala, rotação, translação, cisalhamento e suas composições.
- **Projetiva:** inclui efeito de perspectiva.
- **Não linear:** curvaturas e efeitos elásticos.

Um mapeamento por **triângulos** pode ser afim em cada triângulo, sem que toda a imagem obedeça a uma única transformação global.

Onde usamos deformações e distorções?

- **Jogos e animação:** esticar, dobrar e animar personagens.
- **Edição de imagens:** alinhar rostos, corrigir perspectiva e ajustar objetos.
- **Visão computacional:** registro e alinhamento entre imagens.
- **Mapas e sensoriamento remoto:** retificação de imagens.
- **Medicina:** registrar exames produzidos em momentos diferentes.
- **Interfaces gráficas:** efeitos de transformação de elementos.

Na demonstração OpenGL

Começaremos por uma imagem sintética com grade: linhas retas facilitam perceber se a transformação modificou as proporções.

Transformações afins e composição

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

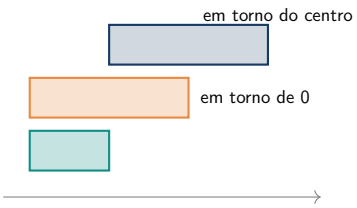
- t_x, t_y: deslocamento; a, d: escala (quando não há rotação/cisalhamento).
- Coeficientes b, c podem introduzir rotação/cisalhamento.
- **A ordem** da multiplicação das matrizes importa.

Retomada

O material de referência trabalha composição de translação, rotação, escala e transformações afins. A seguir usaremos o caso mais visível: escala *somente em X*.

O ponto de referência muda o resultado

- Escalar em X em torno da **origem**: o ponto $x = 0$ fica fixo.
- Escalar em X em torno do **centro**: o centro fica fixo.
- A figura pode “andar” quando mudamos o pivô, mesmo com o mesmo fator.



Escala horizontal: mapa direto e inverso

Mapeamento direto

$$x' = p_x + s_x(x - p_x), \qquad y' = y.$$

Mapeamento inverso (para preencher pixels)

$$x = p_x + \frac{x' - p_x}{s_x}, \qquad y = y'.$$

- No **direto**, alguns pixels de destino podem ficar sem valor.
- No **inverso**, cada posição da saída procura uma posição na origem.
- Usaremos **vizinho mais próximo** para simplificar a amostragem.

Python: escala em X

```python
def deformar_escala_x(imagem, fator=1.4, centro=True):
    h, w, _ = imagem.shape
    resultado = np.full_like(imagem, (243, 245, 250))
    referencia = (w - 1) / 2 if centro else 0
    for y in range(h):
        for x in range(w):
            sx = referencia + (x - referencia) / fator
            ix = int(round(sx))
            if 0 <= ix < w:
                resultado[y, x] = imagem[y, ix]
    return resultado
```

No arquivo base, a função é o **TODO 02**. A versão completa permite comparar o resultado.

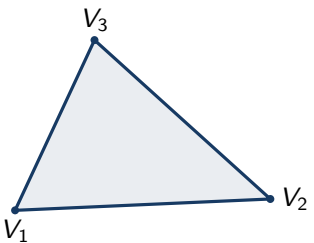
Experimento: deformação em um eixo

Na demonstração OpenGL

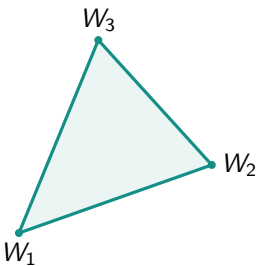
Execute o `codigo completo` da aula. Use **1** para a escala horizontal, **S** para trocar o fator e **O** para alternar pivô origem/centro.

- Teste os fatores 0,65, 1,0, 1,4 e 1,8.
- Compare a imagem com a grade original.
- Qual região fica **fora do enquadramento** quando ampliamos?
- Qual é a diferença visual entre manter a origem e manter o centro?

Deformação bidimensional com triângulos



Triângulo original



Triângulo deformado

Se conhecemos a combinação de um ponto em relação aos três vértices, podemos usar os mesmos pesos no triângulo deformado.

Coordenadas baricêntricas

Representação de qualquer ponto do triângulo

$$P = c_1 V_1 + c_2 V_2 + c_3 V_3, \qquad c_1 + c_2 + c_3 = 1.$$

- Dentro do triângulo, os três pesos são **não negativos**.
- Em um vértice, um peso é 1 e os demais são 0.
- Na aresta entre V_1 e V_2, o peso de V_3 é 0.
- Na deformação, **reutilizamos os pesos**: $P' = c_1 W_1 + c_2 W_2 + c_3 W_3$.

Cálculo manual: mesmo peso, novo ponto

Origem

$$V_1 = (0,0), \quad V_2 = (4,0)$$

$$V_3 = (0,4)$$

$$c = (0{,}25,\ 0{,}25,\ 0{,}50)$$

$$P = (1,2)$$

Destino

$$W_1 = (1,1), \quad W_2 = (6,1)$$

$$W_3 = (1,6)$$

$$P' = (2{,}25,\ 3{,}50)$$

Teste de consistência

A soma dos pesos é 1; o ponto permanece como combinação proporcional dos vértices correspondentes.

Por que mapear do destino para a origem?

1. Para cada pixel P' no triângulo **destino**, encontre c_1, c_2, c_3.
2. Reconstrua $P = c_1 V_1 + c_2 V_2 + c_3 V_3$ no triângulo **origem**.
3. Amostre a cor em P e escreva em P'.

Comparação com o mapeamento direto

Direto: $P \to P'$, podendo deixar buracos. Inverso: $P' \to P$, preenchendo cada pixel de saída coberto pelo triângulo.

Python: calcular os pesos baricêntricos

```
def pesos_baricentricos(x, y, triangulo):
    (x1,y1), (x2,y2), (x3,y3) = triangulo
    d = (y2-y3)*(x1-x3) + (x3-x2)*(y1-y3)
    if abs(d) < 1e-8:
        return None   # Triangulo degenerado
    c1 = ((y2-y3)*(x-x3) + (x3-x2)*(y-y3)) / d
    c2 = ((y3-y1)*(x-x3) + (x1-x3)*(y-y3)) / d
    return c1, c2, 1.0-c1-c2
```

TODO 03: completar a fórmula; **TODO 04:** usá-la em `deformar_triangulo()`.

Python: pixels do triângulo deformado

```
for y in range(h):
    for x in range(w):
        cs = pesos_baricentricos(x, y, destino)
        if cs is None or min(cs) < -1e-5:
            continue  # fora do triangulo
        sx = sum(cs[i] * origem[i][0] for i in range(3))
        sy = sum(cs[i] * origem[i][1] for i in range(3))
        ix, iy = int(round(sx)), int(round(sy))
        if 0 <= ix < w and 0 <= iy < h:
            out[y, x] = imagem[iy, ix]
```

Note que `x,y` indicam o pixel de **destino**, enquanto `sx,sy` referem-se à imagem de **origem**.

Experimento: escolha os triângulos com o mouse

Na demonstração OpenGL

Use o modo **2**. Pressione **P**, clique **3 pontos no painel esquerdo** (origem) e depois **3 no direito** (destino), na mesma ordem dos vértices.

- Compare um triângulo pequeno com outro mais aberto.
- O que acontece se trocarmos a ordem dos pontos?
- Tente três pontos colineares: qual problema matemático aparece?
- Discuta o que é amostragem por **vizinho mais próximo**.

Limitações da deformação por triângulos

- O exemplo preenche **somente a região do triângulo de destino**.
- A imagem fora dele recebe uma cor de fundo; não há transformação de uma malha completa.
- Coordenadas amostradas por arredondamento podem produzir **serrilhamento**.
- Para uma deformação extensa: usar **vários triângulos** conectados.
- Para melhor qualidade: interpolação bilinear, antialiasing e tratamento de limites.

Questão

Como manter deformações contínuas entre triângulos vizinhos? Compartilhar vértices/arestas e manter correspondências consistentes.

O que é preenchimento de regiões?

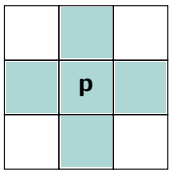
Definição

É identificar os pixels **conectados** a uma semente que atendem a uma condição e atribuir a eles uma cor (ou outro rótulo).

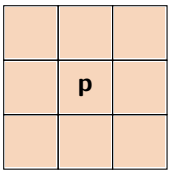
- **Flood fill por região:** segue os pixels com a mesma cor da semente.
- **Boundary fill por borda:** avança até alcançar uma cor que delimita a região.
- A definição de **vizinhança** altera a região alcançada.

4-conectividade e 8-conectividade

4 vizinhos



8 vizinhos



Pergunta

Dois conjuntos que se tocam **apenas na diagonal** são uma única região? Depende da conectividade escolhida.

Flood fill: busca em largura (BFS)

1. Escolher uma **semente** (y, x) e guardar sua cor original.
2. Colocar a semente na **fila**.
3. Retirar um pixel; verificar limites e se já foi visitado.
4. Se a cor corresponde à original, pintar e adicionar vizinhos.
5. Repetir até a fila ficar vazia.

Atenção à implementação

Guardar `visitados` evita repetir pixels. Uma fila iterativa evita estouro de recursão em regiões grandes.

Python: preenchimento por região

```
alvo = imagem[sy, sx].copy()
fila = deque([(sy, sx)])
visitados = np.zeros((h, w), dtype=bool)
while fila:
    y, x = fila.popleft()
    if not (0 <= y < h and 0 <= x < w):
        continue
    if visitados[y, x]:
        continue
    visitados[y, x] = True
    if not np.array_equal(imagem[y, x], alvo):
        continue
    resultado[y, x] = nova_cor
    fila.extend(vizinhos(y, x, conectividade))
```

No código base: **TODO 05**, função `preencher_regiao()`.

Experimento: a conectividade muda a região

Na demonstração OpenGL

Use **3** (4-conectado) e **4** (8-conectado). Clique em uma região colorida **no painel esquerdo**; teste os dois pequenos blocos rosa que se tocam na diagonal.

- O que ocorre com 4-conectividade? E com 8-conectividade?
- Clique no interior do retângulo com obstáculo: a pintura atravessa a parede?
- Pressione **C** para alternar a cor e **R** para restaurar a semente.

Preenchimento por cor vs. preenchimento por borda

Por região (cor-alvo)

- Continua enquanto o pixel tem a **cor inicial**.
- Para se encontrar outra cor, interrompe.

Por borda

- Continua enquanto o pixel **não for da cor de borda** nem da nova cor.
- Pode incluir cores internas diferentes.

Uma borda com abertura pode fazer o preenchimento vazar.

Python: parar na borda

```
# Mesma estrutura de fila/visitados de flood fill.
pixel = imagem[y, x]
if np.array_equal(pixel, cor_borda):
    continue
if np.array_equal(pixel, nova_cor):
    continue
resultado[y, x] = nova_cor
fila.extend(vizinhos(y, x, conectividade))
```

No código base: **TODO 06**, função `preencher_por_borda()`.

A cor da borda do cenário é `(30, 44, 65)`; a saída não altera a imagem original.

Experimento: contornos e preenchimento

Na demonstração OpenGL

Use o modo **5**. Clique dentro da região delimitada e compare com o modo **3**. Pressione **C** para mudar a cor.

- Em região com cores internas diferentes, os dois métodos podem preencher áreas distintas.
- Uma parede de um pixel basta, desde que esteja **fechada** e corresponda à cor de borda.
- Teste clicar **fora** da região delimitada.

Aplicações de preenchimento

- Ferramenta “balde de tinta” em editores gráficos.
- Segmentação e marcação de componentes conectados.
- Seleção de regiões em imagens binárias.
- Identificação de peças, lacunas e furos em imagens industriais.
- Jogos 2D, mapas, labirintos e mapas de ocupação.

Conexão com a atividade

Para classificar peças com 0–3 furos, também precisamos decidir o que é fundo, objeto e região interna conectada.

Checklist final: da teoria à prática

1. Definir deformação e distinguir de transformação rígida.
2. Explicar mapa direto/inverso e o papel do pivô.
3. Calcular pesos baricêntricos: $c_1 + c_2 + c_3 = 1$.
4. Observar 4 vs. 8-conectividade com um contato diagonal.
5. Diferenciar **flood fill** e **boundary fill**.

Na demonstração OpenGL

Compare o código base com o completo nos modos **1 a 5**. O modo **0** é uma revisão opcional de Sobel.

Atividade de aplicação: preenchimento e deformação

- Construa uma nova figura com regiões internas e ao menos um contato diagonal.
- Teste o preenchimento por região em 4 e 8-conectividade.
- Compare com preenchimento por borda usando uma semente interna.
- Aplique escala não uniforme ou deformação triangular e descreva o resultado.

Estudo complementar

As seções **7 a 10** do material original (Transformada de Hough) foram transferidas para um **walkthrough autônomo para os alunos**.

Resumo e referência

- **Deformar:** muda posições e exige regra de amostragem.
- **Preencher:** percorre uma região conectada sujeita a condições de cor/borda.
- **OpenGL:** mostra os resultados processados na CPU com NumPy.

Referência de base: *Deformações e Preenchimento*, José Luis Seixas Junior, UNESPAR (material fornecido: seções iniciais de deformações e preenchimento).